

documentation), currently four target architectures—ARM, MIPS, PowerPC and x86 (32 and 64 bit)—are supported. For each of these, there is support for a QEMU virtual machine, where images can be booted under emulation. For each architecture, there is sample hardware for reference implementation. These are:

- ARM: BeagleBoard
- PowerPC: FreeScale mpc8315e-rdb
- MIPS: Ubiquity Networks Router Station
- x86: Atom Black Sand WEBS-2120

Some of the highlights of Yocto are:

- A new release every six months with the latest software components.
- A repository for open industry specific BSPs.
- Thorough documentation.

The essential tools and components of Yocto

Poky

One of the fundamental components is the reference build system called Poky (originally derived from Poky Linux). Created by Richard Purdie, Poky is a large and complex system, which combines various components such as BitBake (core building engine), Metadata (the config present in the form layers), common generic OE-core, BSPs (interface layers between hardware and OS), and documentation for its build system. Poky's *de facto* tool-chain is based on the GNU Compiler Collection (GCC), but other tool-chains can be used as well.

ADT

Found at <http://www.yoctoproject.org/projects/application-development-toolkit-adt>, this is a toolkit to enable software developers to provide an SDK that can be used by end user developers. ADT has Qemu, build tool-chains, profilers and debuggers as a part of it.

Hob

A new entrant into the project, this tool provides a graphical interface to BitBake. Most tasks that are done on the command-line can be achieved by the Hob GUI. Hob runs as a standalone tool. A new Web-based tool called WebHob is on its way.



 **Note:** There are more tools in the Yocto Project; explore them at <http://www.yoctoproject.org/projects>.

The Yocto build process

The build process consists of fetching the source from various defined sources, patching the code if required, compiling the big bunch, and throwing out image or ROM files. See Figure 2.

Download and try Yocto

Let's get our hands dirty setting up Yocto. Visit <http://yoctoproject.org>, click *Documentation* to keep it handy. Next, bring up a Linux system and install all the necessary dependencies. The

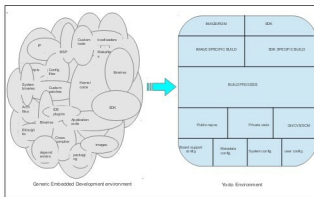


Figure 1: A generic embedded environment transformed to Yocto Project



Figure 2: Yocto build process overview

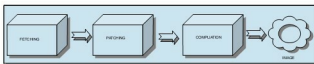


Figure 3: Screenshot of dependencies on an Ubuntu machine

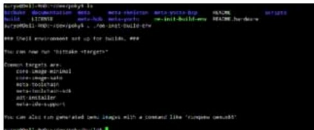


Figure 4: Sourcing the open embedded environment

dependencies for an Ubuntu system are in Figure 3. Refer to your *Quick Start Guide* if you are working on a different distro. Download the latest Yocto release with Git, using the command `git clone git://git.yoctoproject.org/poky.git` to do so. Tarballs are also available.

Explore the main config in the directory `conf/local.conf`.
For a machine with more than one CPU, you can have multiple
compilation threads:

```
BB_NUMBER_THREADS="2"  
PARALLEL_MAKE="-j 2"
```

Set the environment by sourcing the `'oe-init-build-env'` file in the main directory. This file should be sourced (`./oe-init-build-env`) and not executed in a sub-shell. See Figure 4. Once this is done, you are automatically moved to a different directory called `poky/build`.

Execute `bitbake -k core-image-minimal`. This is the minimalist image being created, of the x86 type. By changing options, you can compile for different architectures and different types of images.